

PART VI — THE SELF-LEARNING LAYER

Chapter 32

Learning From Success: The Distillation Engine

“A success is only worth remembering once someone has checked that it actually was one.”

Chapter 32 — Learning From Success: The Distillation Engine

Chapters 30 and 31 built machinery for remembering failure. This chapter builds the mechanism for remembering success — `'self_learner.py'`, the distillation engine that turns a verified, judge-approved interaction into a reusable, independently-retrievable Q&A pair in the `'learned_qa'` collection Chapter 33 will teach the retriever to search.

The chapter's central discipline is narrower than it might first appear: distillation is not "summarize every conversation." It is a strict pipeline — verified interactions only, grounded strictly in the source chunks that supported the original answer, deduplicated by content hash before ever touching the vector store — because Chapter 28.5's warning about memory pollution applies with full force here. An unchecked distillation loop would not accelerate learning; it would launder unverified model output into permanent, confidently-retrieved "knowledge."

By the end of this chapter you will be able to explain why distillation must gate on verified quality rather than distilling every interaction, trace the real `'DISTILL_PROMPT'` this project ships and why each of its constraints exists, and understand how SHA-256 content hashing makes running the same distillation batch twice a safe, idempotent operation rather than a duplication risk.

32.1 The principle: learn only from validated interactions

Definition — Distillation

The process of converting a verified interaction — a query, its judge-approved answer, and the chunks that grounded it — into a compact, reusable Q&A pair stored independently for future retrieval, without introducing any claim absent from the original grounding evidence.

`'run_distillation()'` sources its raw material from exactly one place: `'self.feedback_store.load_good(limit=batch_size)'` — Chapter 29's `'load_good()'` method, filtered to `'quality == "OK"'` records only. This is not an incidental implementation detail; it is the entire safety mechanism. An `'INSUFFICIENT'` or `'USER_THUMBSDOWN'` interaction never reaches `'_generate_qa_pairs()'` at all, because the JUDGE phase's own verdict (Chapter 16.3) or the user's own thumbsdown (Chapter 31) already flagged it as untrustworthy — distillation trusts that upstream gate completely rather than re-deciding quality itself.

This single filtering step is what separates this project's distillation design from the "naive auto-save" ADR-008 explicitly rejected: "save every answer — rejected: hallucinations become permanent." Gating on verified `'OK'` quality is the entire difference between a memory system that compounds genuine successes and one that compounds its own worst mistakes with equal confidence.

It is worth being precise about what "verified" means here, because it is doing a lot of load-bearing work for a single word. `'OK'` quality means the JUDGE phase's `'check_answer_quality()'` found the answer's claims traceable to the retrieved chunks — grounding, not correctness in any absolute sense. A grounded answer can still reflect a corpus that is itself wrong, incomplete, or outdated; distillation inherits whatever quality ceiling the source documents already had. This is not a flaw in the distillation design specifically — it is a reminder that "learn only from verified interactions" verifies faithfulness to evidence, not truth about the world, and the two are not the same guarantee.

32.2 Synthetic Q&A pair generation from verified triples

Each verified interaction contributes a (query, answer, source chunks) triple to ``_generate_qa_pairs()``, which formats the chunks — learned-QA chunks first, tagged ``[LEARNED QA - HIGH PRIORITY]``, then document chunks tagged ``[DOCUMENT]``, the identical precedence ordering Chapter 22C.10 established for context assembly — and sends the whole triple to the LLM via ``DISTILL_PROMPT``. The output is 1-3 synthetic Q&A pairs, each a **rephrasing** of the original question paired with a **self-contained** answer grounded in the same evidence.

Generating multiple rephrasings per interaction, rather than storing the original query verbatim, is a deliberate retrieval-time bet: a future user is far more likely to phrase a related question differently than to type the identical original query, and Chapter 33's hybrid retriever searches ``learned_qa`` by embedding similarity — a collection with three worded variants of the same underlying answer has three separate chances to match a differently-phrased future query, where storing only the original phrasing would have one.

The metadata attached to each stored pair matters as much as the pair's text. ``_upsert_pairs()`` records ``source``, ``original_query``, ``question``, ``answer``, and ``interaction_ts`` alongside the embedded combined text — enough provenance to trace any retrieved learned-QA chunk back to the specific interaction that produced it, which is exactly the audit trail Chapter 28.5.1 argued a responsible memory system needs before it can support review, correction, or deletion of a specific stored fact. A distilled pair with no path back to its origin would be far harder to correct if the source interaction later turned out to be wrong in some way the original quality gate had not caught.

32.3 Strict grounding — no new facts invented

``DISTILL_PROMPT``'s rules are explicit and narrow: "Do not invent facts not present in SOURCE CHUNKS," "Each rephrasing must be semantically distinct," "Answers should be 2-5 sentences." This is the same faithfulness discipline Chapter 22B's LBC compression and merge-validation stages enforce on chunk content, applied here one layer downstream — not to compressed context, but to a **new** synthetic artifact being manufactured specifically to be stored and later trusted as if it were as reliable as a source document.

The stakes for this particular grounding check are arguably higher than any other in the pipeline. A hallucinated claim in a single answer reaches one user, once. A hallucinated claim that survives distillation into ``learned_qa`` becomes retrievable evidence for every future query that happens to match it — and Chapter 22C.10's conflict-resolution rule gives learned QA **precedence** over documents when the two disagree. A fabricated fact laundered through distillation would not just persist; it would outrank the real source material the next time a conflict arose.

Common pitfall — Trusting a judge-approved answer to be trivially re-summarizable

An answer passing ``check_answer_quality()`` proves it was grounded **as originally written**, not that any rephrasing of it will automatically stay grounded. The distillation LLM call is a second generation step with its own fabrication risk, which is exactly why ``DISTILL_PROMPT`` restates the no-invention rule explicitly rather than assuming it's inherited for free.

32.4 The distillation prompt, line by line

```
DISTILL_PROMPT = """You are a knowledge distillation assistant.

Given:
- ORIGINAL QUESTION: {query}
- VERIFIED ANSWER: {answer}
- SOURCE CHUNKS: {chunks}

Your job: produce a JSON array (no markdown, no preamble) of 1-3 objects,
each with:
    "question" : a distinct rephrasing of the original question
    "answer"   : a concise, self-contained answer grounded ONLY in the source
chunks above

Rules:
- Do not invent facts not present in SOURCE CHUNKS.
- Each rephrasing must be semantically distinct (different vocabulary).
- Answers should be 2-5 sentences.
- Output ONLY the JSON array."""
```

Every constraint here maps to a specific downstream need. "1-3 objects" bounds how much a single interaction can grow the collection, preventing one verbose answer from generating dozens of near-duplicate entries. "Semantically distinct" vocabulary directly serves Section 32.2's retrieval-diversity bet — three rephrasings using identical wording would provide no more retrieval coverage than one. "2-5 sentences" keeps each stored pair small enough that Chapter 33's hybrid retrieval does not have to compare oversized learned-QA entries against document chunks of very different scale. "Output ONLY the JSON array" is the identical structured-output discipline Chapter 25.5 and Chapter 26.7 already established — demonstrated by format, not merely described in prose.

32.5 Deduplication with SHA-256 stable IDs

Every candidate pair is combined into a single string — ``f"Q: {q}\nA: {a}"`` — and hashed with SHA-256, truncated to the first 16 hex characters, to produce its ChromaDB entry ID. This is a **content-derived** ID, not a randomly generated or sequentially assigned one: the identical question-answer text always produces the identical ID, regardless of which interaction, which run, or which day generated it.

```
def _stable_id(text: str) -> str:
    return hashlib.sha256(text.encode()).hexdigest()[:16]

combined = f"Q: {q}\nA: {a}"
uid = _stable_id(combined)
...
existing_ids = set(self.collection.get(ids=ids)["ids"])
new_mask = [i for i, uid in enumerate(ids) if uid not in existing_ids]
```

Content-derived IDs make `run_distillation()` naturally idempotent: running the exact same batch twice — whether by accident, a retry, or a deliberate re-run — produces the identical set of hashes both times, and `existing_ids` catches every one of them on the second pass. Figure 32.2 traces this guarantee directly. This is a stronger and simpler property than checking for approximate duplicates by embedding similarity would provide, at the cost of being unable to detect two *differently worded* pairs that happen to express the identical fact — an intentional trade-off, matching Chapter 30.2 and Chapter 31.10's identical exact-match-first reasoning.

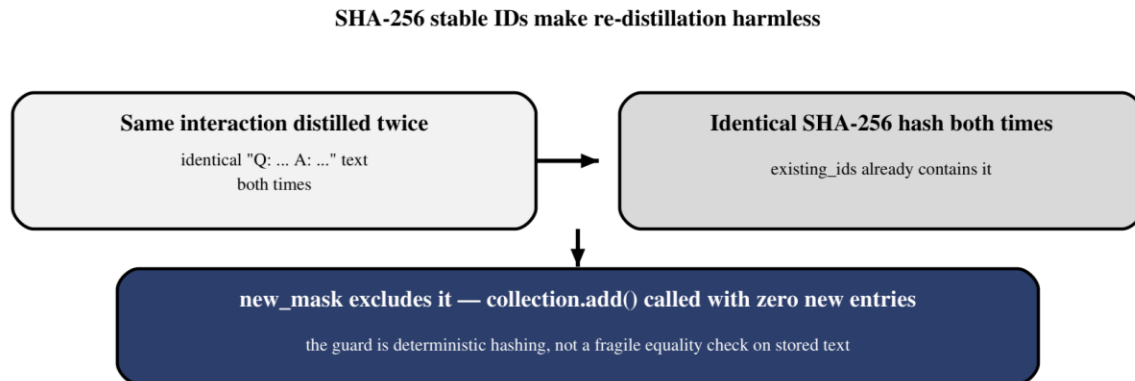


Figure 32.2 — Deterministic content hashing means re-running distillation on the same interaction can never create a duplicate entry.

32.6 Building `self_learner.py`

`SelfLearner` composes an `EmbeddingManager`, an LLM, a `FeedbackStore`, and its own ChromaDB collection handle into one class with a small public surface: `should_learn()` to check the trigger condition, and `run_distillation(batch_size)` to actually perform a pass. Figure 32.1 traces the full internal pipeline `run_distillation()` drives — load, generate, parse, deduplicate, upsert — end to end.

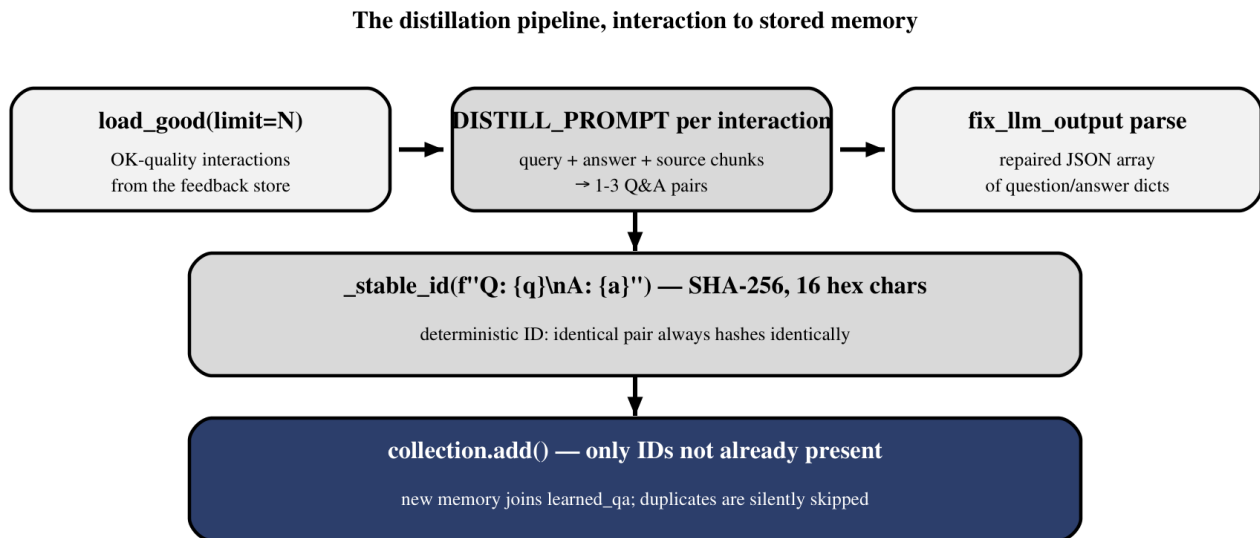


Figure 32.1 — Five stages from a batch of verified interactions to new entries in the `learned_qa` collection, each independently loggable.

`_upsert_pairs()` is where Section 32.5's dedup logic actually lives, and it is worth noting what happens when every candidate pair turns out to already exist: the method logs a clear `'UPSERT SKIPPED'` warning rather than silently returning zero, specifically because a distillation batch that adds nothing is a legitimate, expected outcome (all N good interactions were already distilled in a prior pass) and deserves a different log signature than an actual failure to parse or generate pairs at all — the same operational-

visibility discipline Chapter 22.2's ``_THIN`` separator convention establishes for every other pipeline stage.

The embedding step deserves its own mention because it is where ``SelfLearner`` and the rest of the retrieval pipeline share infrastructure rather than duplicating it. ``self.embedding_manager.generate_embedding(texts)`` is the identical ``EmbeddingManager`` instance every retrieval call already uses, meaning a distilled Q&A pair is embedded with the exact same model, the exact same normalization, and therefore the exact same vector space as every document chunk and every future query it will be compared against. A distillation pipeline that embedded with a different model or configuration would produce vectors that are technically storable but not meaningfully comparable — silently degrading retrieval quality for the entire ``learned_qa`` collection in a way that would be difficult to diagnose after the fact.

``fix_llm_output("distill_qa", raw, llm=self.llm)`` is the same multi-tier JSON repair pipeline Chapter 13B introduced for every other structured LLM call in this project, gated behind the identical ``ENABLE_QA_PAIR_OUTPUT_FIX`` and ``ENABLE_GLOBAL_LLM_OUTPUT_FIX`` flag pair Chapter 36B catalogs. When both flags are off, ``_generate_qa_pairs()`` falls back to ``_parse_to_python(raw)`` directly — a smaller, faster parse path with no LLM-based repair tier, appropriate for a deployment that has already confirmed its distillation model reliably produces clean JSON and does not want to pay the latency cost of a repair call that will rarely fire.

32.7 Triggering distillation

``should_learn()`` implements the simplest of the three trigger strategies book index 32.7 names: ``good % learn_every_n == 0`` — exactly every ``LEARN_EVERY_N`` (5, by this project's default) successful interactions, checked by a plain modulo on the running ``count_good()`` total. This is deterministic and requires no separate scheduler process, at the cost of being purely count-based — a deployment with bursty traffic distills in bursts, and one with steady low traffic distills at a steady, predictable cadence regardless of wall-clock time elapsed.

Strategy	Trigger condition	Trade-off
Every N good interactions (shipped)	<code>count_good() % N == 0</code>	Simple, deterministic; ignores real time elapsed
Time-based	Cron or scheduled interval	Predictable cadence; can fire with zero new material to learn from
Manual (learn command)	User explicitly invokes it	Full control; requires someone to remember to run it

The manual path is not a fallback for when the automatic trigger fails — it is a first-class, independently useful option this project ships as the ``learn`` command (Chapter 35.3), letting an operator force a distillation pass immediately after a batch of known-good interactions rather than waiting for the count to cross a multiple of five. A real, still-open finding from this project's own bug ledger is worth citing honestly here: ``load_good(limit=N)`` returns the **N most recent** good interactions regardless of whether they were already distilled in a prior pass, so a ``learn`` command invoked twice in quick succession, or an automatic trigger firing on a count that re-includes already-processed records, can re-attempt distillation on interactions Section 32.5's hashing will then correctly recognize and skip — safe, but not free, since the LLM calls in Section 32.2 still run before the duplicate is caught.

Common pitfall — Mistaking dedup-safety for efficiency

SHA-256 content hashing guarantees a re-distilled pair never creates a duplicate **entry**, but it does not prevent the **LLM call** that generates that duplicate pair from running again. A watermark or ``distilled: true`` flag on each interaction record — not yet implemented in this project — would be needed to skip the redundant generation call entirely, not merely its storage.

This chapter closes the loop Chapter 28 opened: verified success, distilled without inventing anything beyond its own grounding evidence, deduplicated deterministically, stored as an independently retrievable memory. Chapter 33 picks up exactly where this chapter's output lands — the ``learned_qa`` collection — and asks how a retriever should actually search it alongside the original document corpus.